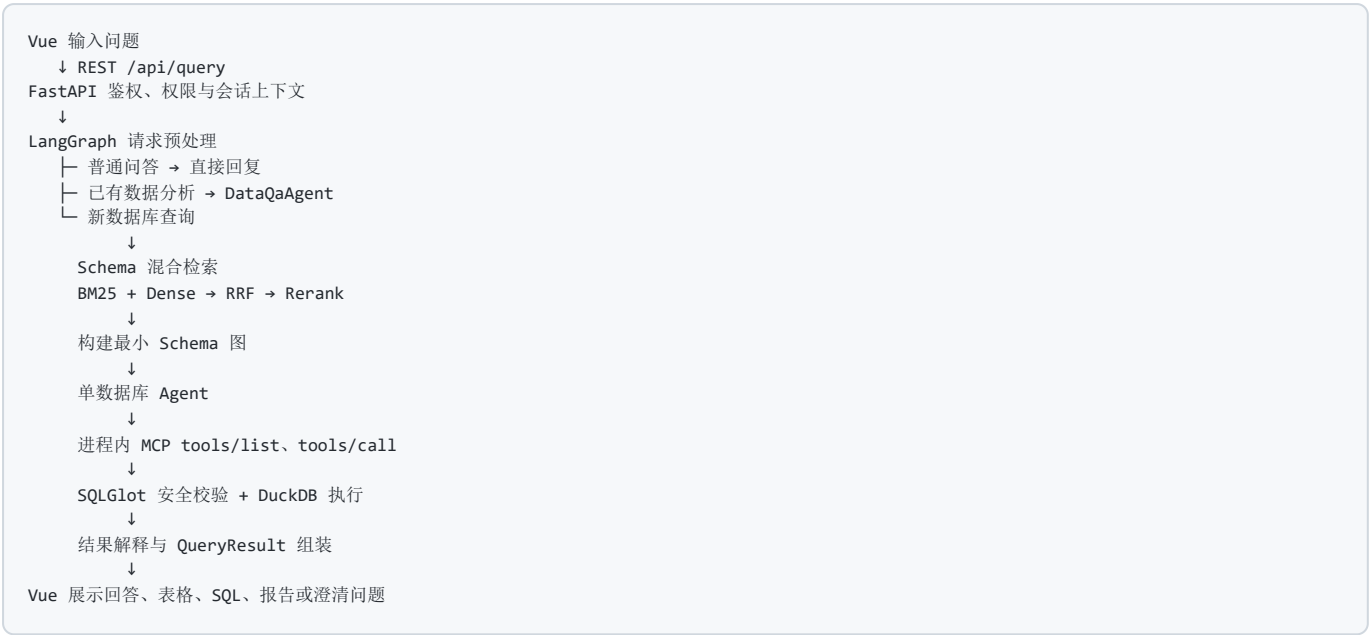


整个系统的主链路可以概括为：



一、用户在前端输入问题

入口在 [App.vue](#)。

用户点击发送或按回车后，`submit()` 会：

1. 清理输入文本。
2. 获取当前对话的 `session_id`。
3. 收集右侧工作区内容：
 - 用户确认的 Schema 字段；
 - 用户选中的历史查询表；
 - 每张历史表最多携带前 50 行。
4. 调用 `api.query()`。
5. 等待后端完整返回结果。
6. 将结果加入当前对话并保存前端会话状态。

请求结构大致是：

```
{
  "query": "贵州茅台最近五年的净利润是多少？",
  "session_id": "conversation-id",
  "workspace": {
    "schema_fields": [],
    "analysis_table_ids": [],
    "analysis_tables": []
  }
}
```

[api.ts](#) 会：

- 从 `sessionStorage` 读取登录 Token；
- 添加 `Authorization: Bearer ...`；
- 发送 `POST /api/query`；
- 查询请求最长等待 300 秒；
- 当前没有流式输出，前端会一直等待整个 `QueryResult` 返回。

二、FastAPI 接收请求

入口是 `routes.py`。

后端首先执行：

1. Bearer Token 鉴权。
2. 游客查询次数检查。
3. 将 Pydantic `WorkspaceConfig` 转换成字典。
4. 调用：

```
service.submit(
    payload.query,
    payload.session_id,
    workspace,
    user.user_id,
)
```

API 输入输出契约定义在 `models.py`。最终统一返回 `QueryResult`，其中可能包含：

- `status`：完成、失败或等待澄清；
- `route`：数据库查询、已有数据分析或直接回答；
- SQL；
- 表格列和数据；
- 文字分析；
- Schema 检索过程；
- MCP 工具调用记录；
- 报告和图表配置；
- 澄清选项。

三、Service 准备用户权限和会话上下文

核心入口是 `finquery_service.py`。

`FinQueryService.submit()` 会：

1. 根据 `user_id` 解析用户可访问的数据库和表。
2. 把用户 ID 拼入会话 ID，防止不同用户共享上下文：

```
user_id:session_id
```

3. 检查上一轮是否正在等待澄清。
4. 如果当前输入是“第一个”“沪深300”“选二”等澄清答案，直接恢复原任务。
5. 否则生成新的 `task_id`。
6. 规范化前端传来的 `workspace`。
7. 保存用户消息。
8. 构造 LangGraph 初始状态并执行工作流。

工作流初始状态在 `finquery_service.py` 中构造，包含：

```
query
session_id
workspace
access_scope
近期轻量对话
短期记忆和摘要
最近一张查询结果
用户选中的历史表格
execution_log
```

这里的 `workspace` 是查询上下文，不是磁盘工作目录。

四、LangGraph 首先判断问题属于哪条路线

工作流定义在 `query_graph.py`。

第一节点是 `preprocess`。`RequestPreprocessor` 用一次模型调用完成：

- 是否依赖上文；
- 是否需要改写成独立问题；
- 意图分类；
- Schema 检索词提取。

代码在 `preprocessing.py`。

它把请求分成三种：

1. `direct_response`

适用于：

- 闲聊；
- 能力询问；
- 不需要数据库的问题；
- 无法判断用户到底想查新数据还是分析旧结果；
- 模型需要先自然语言追问。

这条路线不会检索 Schema，也不会调用数据库，预处理模型产生的回复直接被包装成 `QueryResult`。

2. `data_qa`

适用于：

- “解释一下刚才的结果”
- “对比这两张表”
- “基于刚才的数据生成分析报告”

它只分析已有查询结果，不重新查数据库。

3. `database_query`

适用于：

- 查询指标；
- 获取明细；
- 排名、统计、趋势或对比；
- 需要从金融数据库获取新数据。

只有这条路线才进入 Schema 检索和 NL2SQL。

如果预处理模型不可用，系统采取保守策略：返回“模型服务暂时不可用”，不会冒险查询数据库。

五、数据库查询首先进行字段级 Schema 召回

入口在 `query_graph.py`。

1. 加载或构建 Schema 索引

`SchemaIndex` 会先检查本地缓存：

```
backend/data/schema_store.<database-signature>.json
```

如果缓存不存在、Schema 变化、同义词变化或 Embedding 模型变化，则：

1. 为每个字段生成检索文档；
2. 调用 Embedding 模型生成向量；
3. 保存到本地 JSON 索引。

2. 权限过滤

只有当前用户有权访问的表字段才能参与检索。权限控制不是最后才执行，而是贯穿：

Schema 可见性
→ Schema 召回
→ Schema 图
→ MCP 工具发现
→ DuckDB 执行

3. 混合召回

当前实现按以下顺序检索：



具体实现在 [retrieval/service.py](#)。

需要特别说明：

- 当前 Dense 检索是本地向量逐个点积比较；
- 没有使用 HNSW 或 IVF；
- 没有连接 Milvus；
- RRF 使用 $1 / (60 + \text{rank})$ 融合 BM25 和 Dense 排名；
- 最终由独立 Rerank 模型重新打分。

用户在 workspace 中明确确认的字段会以 `score=1.0` 强制加入结果，但仍要通过权限检查。

六、将召回字段构造成 Schema 图

代码位于 [graph.py](#)。

Schema 图包含：

- 相关数据表；
- 召回字段；
- 实体名称和代码字段；
- 表之间的关联关系；
- 为连接表补充的关联键；
- 图版本。

如果召回了多个表，系统会根据 `_schema.json` 中声明的关系寻找最短连接路径，补齐 JOIN 所需字段，而不是让模型凭经验猜关联关系。

最终生成类似：

```
表 stock_daily
- stock_daily.symbol
- stock_daily.stock_name
- stock_daily.trade_date
- stock_daily.close

表 financial_statement
- financial_statement.symbol
- financial_statement.report_date
- financial_statement.net_profit

关联:
stock_daily.symbol = financial_statement.symbol
```

这段严格限定的 Schema 文本随后提供给 SQL Agent。

七、单数据库 Agent 规划并执行查询

当前启用的是单数据库路径。入口在 [single_database_agent.py](#)。

Agent 会收到：

- 用户独立问题；
- 目标数据库；
- Schema 图；
- 召回字段；
- 低置信度候选；
- 用户确认字段和参数；
- 当前可用 MCP 工具。

MCP 工具发现

Agent 每次请求都会调用：

```
tools/list
```

然后根据数据库查询 Skill 的允许列表过滤工具：

```
current_datetime
resolve_date_range
query_<database>
```

例如当前数据库对应：

```
query_trade_data
```

模型决策

模型每轮只能返回两种动作：

```
call_tool
clarify
```

如果信息充分，它生成：

```
{
  "action": "call_tool",
  "tool_name": "query_trade_data",
  "arguments": {
    "sql": "SELECT ..."
  },
  "query_contract": {
    "grain": "时间趋势",
    "source_tables": ["financial_statement"],
    "filters": [],
    "metrics": ["净利润"],
    "output_columns": ["报告期", "净利润"]
  }
}
```

如果业务口径缺失且确实会改变查询结果，它返回澄清问题。

八、当前 MCP 是进程内 MCP，不是远程 MCP

这是本项目架构中一个很重要的边界。

[query_graph.py](#) 创建：

```
LocalMcpClient(
    create_local_mcp_server(...)
)
```

因此当前链路是：

```
Vue
↓ REST
FastAPI / LangGraph
↓ 进程内 MCP
本地 MCP Server
↓
DuckDB
```

不是：

```
浏览器 → 远程 MCP Server
```

`LocalMcpClient` 使用 MCP SDK 完成 `list_tools()` 和 `call_tool()`，但 Client、Server、FastAPI 都运行在同一个 Python 进程中。[client.py](#)

MCP Server 根据当前用户权限动态注册数据库工具。[server.py](#)

九、需要澄清时，LangGraph 暂停

如果 Agent 认为必须确认口径，会进入：

```
human_clarification
```

代码在 [query_graph.py](#)。

LangGraph 通过 `interrupt()` 暂停当前任务，返回：

```
{
  "status": "waiting_clarification",
  "clarification": {
    "question": "你希望查询净利润还是归母净利润？",
    "options": [...]
  }
}
```

前端显示问题和选项提示，但当前界面没有直接调用 `api.clarify()` 的按钮逻辑。用户是在输入框中回复选项。

下一次仍然发送普通的 `/api/query`。Service 发现当前会话有待澄清任务后，将“第一个”“选二”或选项标签匹配成 `option_id`，然后使用：

```
Command(resume={"option_id": option_id})
```

恢复同一个 LangGraph 任务。

确认结果写入：

```
confirmed_schema_tables
confirmed_parameters
```

之后重新进入 Schema 检索。

需要注意：LangGraph checkpoint 当前使用 `InMemorySaver`，所以等待澄清的活动任务依赖当前后端进程；重启后不能靠会话归档恢复这个中断点。

十、通过 MCP 调用 DuckDB

数据库 MCP Tool 最终调用 [database_tools.py](#)：

```
engine.execute(database, sql, access_scope)
```

`DuckDbEngine` 的执行过程在 [duckdb_engine.py](#)。

执行前检查

SQLGlot 会解析 SQL，并检查：

- 只能有一条语句；
- 只能是 `SELECT` 或 `WITH` ；
- 禁止增删改、DDL、权限和管理操作；
- 禁止 `SELECT *` ；
- 禁止未知表；
- 检查用户是否有权访问数据库和表；
- 单表查询时检查未知字段。

数据加载

每次查询创建一个内存 DuckDB 连接，把数据文件注册成只读 View：

```
CSV                → read_csv_auto()
单个 Parquet       → read_parquet()
Parquet 分区目录   → read_parquet("*.parquet")
```

返回数据

DuckDB 最多读取 201 行，但只返回前 200 行，用于执行结果截断。

如果 SQL 执行失败，Agent 最多可以根据真实数据库错误修正 SQL；MCP 工具调用总次数最多为 3 次，并禁止原样重复已经失败的 SQL。

十一、对查询结果生成最终说明

SQL 成功后，[response_generator.py](#) 再调用一次模型，输入：

- 原始问题；
- 实际执行的 SQL；
- 返回列；
- 真实结果数据；
- Schema 上下文；
- 用户选中的历史分析表。

模型只负责生成：

```
{
  "valid": true,
  "reason": "结果可以回答问题",
  "title": "贵州茅台净利润趋势",
  "analysis": "....."
}
```

它不能重新计算或编造结果中不存在的数值。

如果这一阶段模型失败，已经成功执行的 SQL 和结果表不会丢失。系统仍返回表格，只把说明改成：

SQL 已成功执行，但结果说明生成失败

十二、已有结果分析路线

如果预处理器判断为 `data_qa`，则进入 `data_qa_agent.py`。

它使用：

- 短期对话记忆；
- 最近一次查询结果；
- 用户拖入工作区的历史表格。

这条路线明确禁止重新查询数据库。

普通解释直接生成文字；只有用户明确要求“报告”时才生成：

- Markdown 报告；
- 柱状图配置；
- 饼图配置。

图表工具不会计算新数据，只能引用已有结果真实存在的 `task_id` 和列名。

十三、返回前端并展示

所有分支最终由 `result_builder.py` 转换为统一 `QueryResult`。

Service 随后：

1. 将任务保存到当前进程的会话状态；
2. 可选写入 SQLite 会话归档；
3. 保存 Assistant 消息；
4. 异步触发短期记忆摘要；
5. 通过 FastAPI 返回 JSON。

前端根据 `route` 和 `status` 展示不同内容：

- `waiting_clarification`：澄清问题；
- `direct_response`：普通文字回答；
- `data_qa`：结果解释或 Markdown 报告；
- `database_query`：结果标题、解释、表格、SQL、召回详情和执行信息。

渲染入口可见 [App.vue](#)。

最终调用链

正常数据库问数的精确调用顺序是：

```
App.vue.submit()
→ api.query()
→ POST /api/query
→ require_user()
→ FinQueryService.submit()
→ SessionContext 构造上下文
→ QueryWorkflow.invoke()
→ RequestPreprocessor.prepare()
→ SchemaIndex.retrieve()
→ BM25 + Dense + RRF + Rerank
→ SchemaGraphBuilder.build()
→ SingleDatabaseAgent.prepare()
→ LocalMcpClient.list_tools()
→ 模型生成 SQL / 选择工具
→ LocalMcpClient.call_tool()
→ query_trade_data()
→ DuckDbEngine.execute()
→ SQLGlot 校验
→ DuckDB 执行
→ ResponseGenerator.finalize()
→ ResultBuilder.completed()
→ FinQueryService 保存会话
→ FastAPI 返回 QueryResult
→ Vue 展示回答和表格
```

我没有修改任何项目文件。这次检查还确认了两个容易产生误解的事实：当前 MCP 是后端进程内 MCP，而不是远程 MCP；当前 Dense Schema 检索是本地全量向量点积，不是 HNSW、IVF 或 Milvus。